

MATERIA:

- Python para Ciencia de Datos

Instancia: Primer Parcial

Docente: Juan Pablo Rigotti

Fecha: 16/09/2026

NOMBRE Y APELLIDO:	LU:
Carrera:	Cantidad de hojas entregadas:

Consignas para desarrollo del examen

1. Lea atentamente cada una de las preguntas para asegurarse de responder exactamente lo que se solicita.
2. Piense y elabore su respuesta de forma tal que la misma sea clara y concisa.
3. Se evaluará tanto el conocimiento como la claridad de la exposición (incluida ortografía).
4. La interpretación forma parte del examen y de su nota final.
5. Cada pregunta tiene su puntuación siendo la nota final definida por la tabla adjunta.
6. **No está permitido el uso del celular ni de ningún otro artefacto tecnológico durante el examen (salvo calculadora) y los mismos deben estar guardados debidamente al frente o fondo del aula.**
7. **No está permitido consultar libros, cuadernos o apuntes durante el examen. Recuerde que la honestidad académica contribuye a su formación personal y si la infringe, recursará la materia recibiendo sanción.**

Condiciones de aprobación: Nota 4, debe contestar correctamente como mínimo el 60% de las preguntas formuladas.

Duración del examen: 120 minutos.

Escala de Notas:

Puntaje	0 - 59	60 - 64	65 - 69	70 - 74	75 - 79	80 - 87	88 - 94	95 - 100
Nota	Desaprobado	4	5	6	7	8	9	10

Parte 1

Dada una lista de diccionarios con los siguientes 3 registros, responder las preguntas:

```
# Crear una lista de diccionarios con los siguientes datos
unis = [
    {"nombre": "Stanford", "ranking": 4, "emp_ratio": 97.8},
    {"nombre": "MIT", "ranking": 1, "emp_ratio": 96.2},
    {"nombre": "Yale", "ranking": 9, "emp_ratio": 93.3},
]

# nombre: es el nombre de la institución educativa
# ranking : es la posición que ocupa en un ranking Forbes 2026
# emp_ratio: es el porcentaje de graduados empleados dentro del primer año de egreso
```

Pregunta 1 [Desarrollo] (____ / 10 Puntos)

Se ha desarrollado una función que muestre en consola/terminal los diccionarios de la lista unis:

```
# Declaración de la función
def mostrar_unis(unis):
    for u in unis:
        print(u)

# Llamado/invocación de la función:
mostrar_unis(unis)
```

Refactorizá la función para que imprima sólo los primeros n registros que se pasan como argumento. Se provee código de soporte:

```
def mostrar_unis(unis, cant_reg=None):
    if cant_reg and cant_reg <= len(unis):
        selected_unis = unis[:cant_reg]
    else:
        selected_unis = unis
    for u in selected_unis:
        print(u)

# Llamado/invocación de la función:
mostrar_unis(unis, 6)
```

Pregunta 2 [Desarrollo] (____ / 10 Puntos)

Se solicita desarrollar la función calificar_unis() de manera que agregue a cada diccionario un par clave:valor. Siendo la clave “nivel” y el valor:

- “Top5” si el valor de “emp_ratio” es menor o igual a 5
- “Top10” si el valor de “emp_ratio” es menor o igual a 10
- “Top50” para cualquier otro valor de “emp_ratio”

```
# Calificar universidades
def calificar_unis(unis):
    for u in unis:
        if 1 <= u["ranking"] <=5:
            u["nivel"] = "Top5"
        elif 5 < u["ranking"] <=10:
            u["nivel"] = "Top10"
        else:
            u["nivel"] = "Top50"
```

```
# Llamado/invocación de la función:  
calificar_unis(unis)
```

Pregunta 3 [Desarrollo] (____ / 10 Puntos)

Se ha desarrollado la función **uni_mejor_emp_ratio()** de manera que al invocarla y pasarle como argumento la lista de diccionarios **unis** retorna:

```
uni_mejor_emp_ratio(unis)
```

*La Universidad con mayor ratio de empleabilidad es **AAA** que se encuentra en el top **BBB** con un índice de empleabilidad de **CCC** % sobre un promedio de **DDD** %.*

Completá el código que falta en **buscar_mejor_emp_ratio()**:

```
# Genera una funcion que retorne el diccionario de la universidad con mayor emp_ratio  
  
def buscar_mejor_emp_ratio(unis):  
    mayor_emp_rate = unis[0]  
    for u in unis:  
        if u["emp_ratio"] > mayor_emp_rate["emp_ratio"]:  
            mayor_emp_rate = u  
    return mayor_emp_rate
```

Completá el código que falta en **calcular_emp_ratio_avg()**:

```
# Calcula el porcentaje de empleabilidad promedio (promedio de emp_ratio)  
  
def calcular_emp_ratio_avg(unis):  
    emp_list = [u["emp_ratio"] for u in unis]  
    emp_ratio_avg = sum(emp_list) / len(emp_list)  
    return emp_ratio_avg
```

Agregá a la función **uni_mejor_emp_ratio()** manejo de excepciones (**try/except**) para capturar errores y mostrarlos en consola/terminal:

Invoca a las funciones auxiliares para generar la etiqueta de nivel con el Top5, Top10, buscar el diccionario con el mejor empratio y calcular el emp_ratio_avg. Luego muestra el resultado en consola/terminal

```
def uni_mejor_emp_ratio(unis):  
    try:  
        calificar_unis(unis)  
        uni_mr = buscar_mejor_emp_ratio(unis)  
        er_avg = calcular_emp_ratio_avg(unis)  
        print(  
            f"La Universidad con mayor ratio de empleabilidad es {uni_mr['nombre']} que se  
            encuentra en el {uni_mr['nivel']} con un índice de empleabilidad de  
            {uni_mr['emp_ratio']} % sobre un promedio de {er_avg}"  
        )  
    except Exception as e:  
        print(f"ERROR: {e}")
```

Pregunta 4 [Desarrollo] (____ / 10 Puntos)

Se ha desarrollado la función `calcular_est_emp_unis()` que calcula las principales medidas estadísticas (de posición y dispersión), la cuales se imprimen y muestran en el segundo bloque de código:

Calcula las medidas estadísticas del campo emp_ratio de unis

```
def calcular_est_emp_unis(unis):  
    emp_list = [u["emp_ratio"] for u in unis]  
    emp_array = np.array(emp_list)  
    count = len(emp_array)  
    max = np.max(emp_array)  
    min = np.min(emp_array)  
    mean = np.mean(emp_array)  
    median = np.median(emp_array)  
    std = np.std(emp_array)  
  
    print(f"Cantidad: {count}\nMáximo: {max}\nMínimo: {min}\nMedia: {mean}\nMediana:  
{median}\nDesvío: {std}")
```

```
cal_est_emp_unis(unis)
```

```
Cantidad: 3  
Máximo: 97.8
```

Mínimo: 93.3
Media: 95.77
Mediana: 96.20
Desvío: 1.86

Explicá c/u de las líneas del código de la función. Finalmente, indicá la diferencia de trabajar con listas nativas de Python y con Arrays de NumPy.

Parte 2

En esta segunda parte, se carga el Dataset **US_Top_50_Universities_2026** descargado de Kaggle y se imprime el shape:

```
# Cargar el dataset localmente
df = pd.read_csv("US_Top_50_Universities_2026.csv")

(53, 8)
```

Pregunta 5 [Desarrollo] (____ / 10 Puntos)

Explicá que información relevante provee el método info() del dataframe de Pandas df:

```
df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 53 entries, 0 to 18
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   University_Name        53 non-null    object
1   National_Rank          53 non-null    int64
2   Founded_Year           53 non-null    int64
3   Institution_Type        53 non-null    object
4   State                  53 non-null    object
5   Research_Impact_Score  53 non-null    float64
6   Intl_Student_Ratio     48 non-null    float64
7   Employment_Rate        53 non-null    object
dtypes: float64(2), int64(2), object(4)
memory usage: 5.8+ KB
```

El dataset posee 53 registros y 8 columnas.

Para cada columna se indica su nombre, la cantidad de registros no-nulos y el tipo de dato (que interpreta Python)

Por ejemplo, para University_Name no hay registros nulos, el dtype es object, correcto.

En cambio, para Intl_student_ratio hay 5 registros nulos, dtype float también correcto.

Employment_rate por algún motivo asume que es Object, cuando al parecer debería ser float, al igual que Intl_student_ratio

Pregunta 6 [Desarrollo] (____ / 10 Puntos)

Se ejecuta el método **describe()** para explorar las variables numéricas y categóricas del Dataframe

df.describe()				
	national_rank	founded_year	research_impact_score	intl_student_ratio
count	53.000000	53.000000	53.000000	47.000000
mean	25.698113	1845.886792	80.250943	44.976596
std	14.401636	63.714405	23.822498	31.179356
min	1.000000	1636.000000	28.500000	5.100000
25%	14.000000	1819.000000	82.100000	15.200000
50%	26.000000	1861.000000	90.100000	46.200000
75%	38.000000	1885.000000	96.300000	73.100000
max	50.000000	1965.000000	100.000000	94.200000

df.describe(include="object")				
	university_name	institution_type	state	employment_rate
count	53	53	53	53
unique	50	2	23	47
top	Northwestern University	Private	CA	86,4

Explicá:

1. Que conclusiones se pueden observar de la variable intl_student_ratio correspondiente al porcentaje de estudiantes internacionales.
 - a. 47 registros
 - b. Avg 44.9
 - c. Desvio std: 31.17
 - d. Min-Max, Q1, Q2, Q3 (Mediana)

2. Porque employment_rate, que es la variable equivalente a emp_ratio de la parte 1, aqui se ve como categórica?
- a. Porque en la primera parte, el separador decimal es el punto y aquí en la coma.
3. Que se puede concluir de las variables categóricas?
- a. Hay 50 universidades, Northwestern es la top (pero por los registros duplicados)
- b. Solo 2 tipos de instituciones (Private es uno de ellos)
- c. 23 estados (habría más universidades en CA)
- d. Employment_rate no es representativo, ya que por el separador de mil (comma) esta incorrectamente etiquetada.

Pregunta 7 [Desarrollo] (____ / 10 Puntos)

Que significa la salida del siguiente código?

```
df.duplicated().sum()
np.int64(3)
```

Indica que hay 3 registros duplicados

Sobre la visualizacion de registrs duplicados, cual es correcta? Justifica tu respuesta:

- A. `df[df.duplicated()].sort_values(by="university_name")`
- B. `df[df.duplicated(keep=False)].sort_values(by="university_name")`
- C. `df[df.duplicated(keep="first").sort_values("university_name", ascending=False)`
- D. `df[df.drop_duplicates()].sort_values(by="university_name")`

Pregunta 8 [Desarrollo] (____ / 10 Puntos)

Explicá que hace la siguiente línea de código:

```
# Reemplazar , por . en employment_rate
df["employment_rate"] = df["employment_rate"].str.replace(",", ".", regex=False)
```

Reemplaza (,) por (.) en cada valor de employment_rate para poder convertir de str a float

Sobre la conversión de tipo de dato str a float, cuál de las siguientes es correcta? Justificá tu respuesta.

- A. `df["employment_rate"] = pd.to_numeric(df["employment_rate"], errors="ignore")`
- B. `df["employment_rate"] = pd.to_numeric(df["employment_rate"], errors="coerce")`

- C. `df["employment_rate"] = df["employment_rate"].astype(float, errors="coerce")`
D. `df["employment_rate"] = pd.to_numeric(df["employment_rate"], errors="raise")`

Justificación:

- `to_numeric` es un método de Pandas y admite "raise" y "coerce" para el parámetro `errors`
- `astype` es un método del dataframe y admite "raise" y "ignore" para el parámetro `errors`

Pregunta 9 [Desarrollo] (____ / 10 Puntos)

El siguiente método retorna:

```
df.isnull().sum()
intl_student_ratio    5
employment_rate      1
dtype: int64
```

Se decide eliminar registros nulos en el campo **employment_rate**, pero imputar nulos en `intl_student_ratio`.

Completá la función para eliminar nulos:

```
# Eliminar nulos en employment_rate
df.dropna(subset="employment_rate", inplace=True)
```

Justificación:

- `subset`: indica que columna evalúa para eliminar el registro
- `inplace=True` es necesario porque `dropna` no modifica el df original

Y respondé cuál de los siguientes opciones es correcta para imputar nulos en `intl_student_ratio`?

```
df["intl_student_ratio"] = df["intl_student_ratio"].fillna(df["intl_student_ratio"].median())
df.fillna(subset="intl_student_ratio", inplace=True)
```

Justificación:

- `fillna` no posee método `subset`

Pregunta 10 [Desarrollo] (____ / 10 Puntos)

Considerando:

- `national_rank` : es el 2026 Forbes ranking position (1-50) donde [1 Mayor Importancia - 0 Menor Importancia]
- `employment_rate`: % of graduados empleados dentro del año de graduación

Que interpretás del siguiente código? Explicá lo más que puedas:

```

df["ranking"] = 51 - df["national_rank"]

df["ranking_norm"] = (
    df["ranking"].max() - df["ranking"]
) / (
    df["ranking"].max() - df["ranking"].min()
)

df["employment_norm"] = (
    df["employment_rate"] - df["employment_rate"].min()
) / (
    df["employment_rate"].max() - df["employment_rate"].min()
)

df["recommendation_index"] = (
    0.6 * df["ranking_norm"] +
    0.4 * df["employment_norm"]
)

recommendation = df.sort_values(
    by="recommendation_index",
    ascending=False
).head(5)

recommendation[["university_name", "national_rank", "employment_rate",
"recommendation_index"]]

```

	university_name	national_rank	employment_rate	recommendation_index
42	Carnegie Mellon University	43	93.7	0.811141
45	University of Texas at Austin	46	89.2	0.734668
47	Colgate University	48	88.1	0.731485
48	New York University (NYU)	49	87.4	0.726120
40	Georgetown University	41	90.3	0.701117

Explicación del código:

- national_rank [50 mejor – 0 peor]
- ranking [1 mejor – 50 peor]
- ranking_norm [1 mejor – 0 peor]
- employment_rate [100 mejor – 0 peor]
- employment_norm [1 mejor – 0 peor]
- recommendation_index = 0.6 * ranking_norm + 0.4 * employment_norm

Se ordena el df por recommendation_index de mayor a menor y se filtran los primeros 5 registros para generar el listado de Universidades sugeridas.